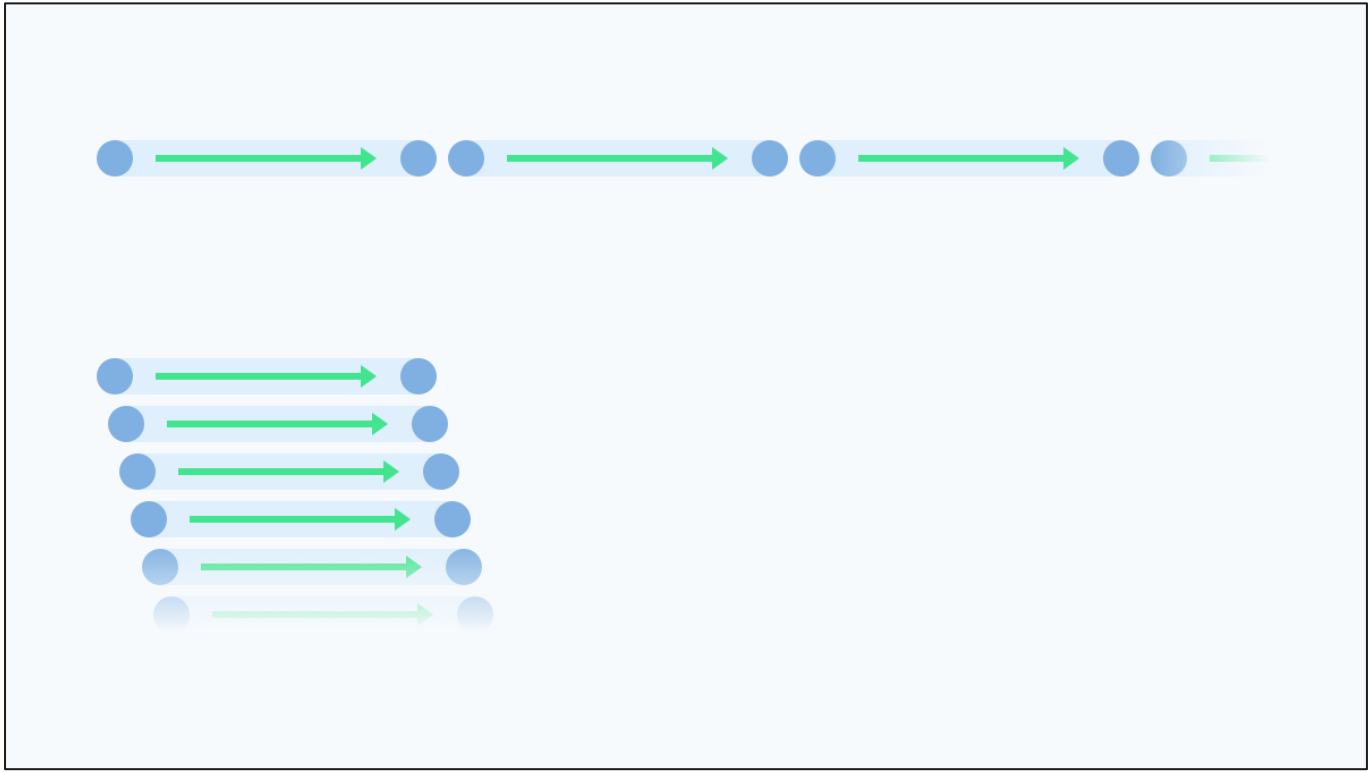
CS 4032 – Web Programming

National University
Of Computer and Emerging Sciences

Asynchronous Programming and Promises

Synchronous vs Asynchronous



Synchronous vs Asynchronous

```
(function() {
  ...
  function init() {
    console.log('page loaded');
    qs('button').addEventListener('click', clickHandler);
    showMenu();
  }

  function showMenu() {
    id('menu').classList.remove('hidden');
  }

  function clickHandler() {
    /* Your code */
  }
})();
```

Callbacks?

Callbacks are a very powerful feature in event-driven programming.

It's useful to have the ability in the JavaScript language to pass callback functions as arguments to other functions like `addEventListener` and `setTimeout` in JS

Asynchronous Programming

The JS programs we've been writing are naturally asynchronous

We pass functions as arguments to other functions so that we can 'call back later' once we know something we expect occurred.

We've already been writing asynchronously!

```
btn.addEventListener('click', callbackFn);
btn.addEventListener('click', function() {
  ...
});
btn.addEventListener('click', () => {
  ...
});
```

```
setTimeout(callbackFn, 2000);
setTimeout(function() {
  ...
}, 2000);
setTimeout(() => {
  ...
}, 2000);
```

Why is JavaScript so different?

Java, and other compiled languages, are often used to build *systems*.

- Objects are great to compose together to build complex systems.
- Systems must be reliable - a benefit of strict types, compiling, and well-defined behavior in Java.

JavaScript is used to *interact* and *communicate*.

- It listens.
- It responds.
- It requests.

Whereas in Java, programs often have a well-defined specification (behavior), JS has to deal with uncertainty (weird users, unavailable servers, no internet connection, etc.)

What if?

```
let myBtn = qs('button:nth-child(1)');
while (!myBtn.clicked) {
  // twiddle our thumbs
}
console.log('"Finally Been Clicked", starring Anne Hathaway');

let myBtn2 = qs('button:nth-child(2)');
while (!myBtn2.clicked) {
  // twiddle our thumbs
}
console.log('"Click 2", never coming soon to a theater near you');
```

- This won't work (and will crash your browser)
- We wouldn't be able to do anything while we were waiting
- But the synchronous logic is nice
- What if we could could make our code feel more synchronous?

Analogy: You're out for pizza

At the restaurant you might follow these steps:

- Request menu
- Order pizza
- Check pizza
- Eat pizza
- Pay for pizza

Each step can't continue before the previous finishes.

What do you do in between?

```
requestMenu();
// twiddle thumbs
orderPizza();
// twiddle thumbs
verifyPizza();
// twiddle thumbs
eatPizza();
// twiddle thumbs
payForPizza();
```

Callback (again) to Callbacks:

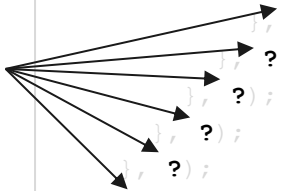
We can imagine all of these steps as a series of callbacks, depending on the event previous to them:

```
function order() {
  setTimeout(function() {
    makeRequest('Requesting menu...');
    setTimeout(function() {
      makeRequest('Ordering pizza...');
      setTimeout(function() {
        makeRequest('Checking pizza...');
        setTimeout(function() {
          makeRequest('Eating pizza...');
          setTimeout(function() {
            makeRequest('Paying for pizza...');
            setTimeout(function() {
              let response = makeRequest('Done! Heading home.');
              console.log(response);
            }, ?);
          }, ?);
        }, ?);
      }, ?);
    }, ?);
  }, ?);
}
```

Callback (again) to Callbacks:

```
function order() {
  setTimeout(function() {
    makeRequest('Requesting menu...');
    setTimeout(function() {
      makeRequest('Ordering pizza...');
      setTimeout(function() {
        makeRequest('Checking pizza...');
        setTimeout(function() {
          makeRequest('Eating pizza...');
          setTimeout(function() {
            makeRequest('Paying for pizza...');
            setTimeout(function() {
              let response = makeRequest('Done! Heading home.');
              console.log(response);
            }, ?);
          }, ?);
        }, ?);
      }, ?);
    }, ?);
  }, ?);
}
```

But how long should we wait before running each step?



Wouldn’t it be Nice...

... if we could do this?

```
orderPizza()
  .then(verify)
  .then(eat)
  .then(pay)
  .catch(badPizza);
```

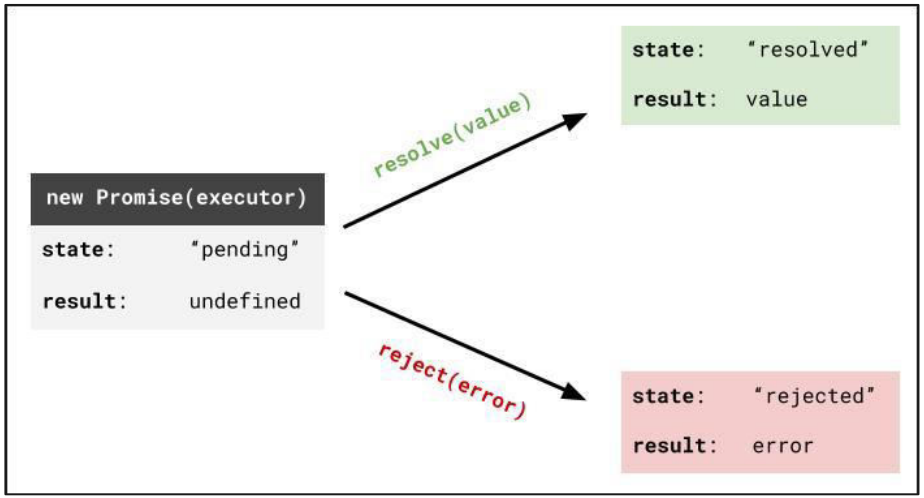
Uncertainty

Some operations take an unknown amount of time or have a not insignificant chance of failure

- File I/O
- Database transactions
- HTTP requests
  - Resource doesn't exist (404)
  - Really long response time or server is down
  - Bad internet connection

Whether these operations succeed or fail, we still want to do something in response

Promises



Promises are a sort of contract:

- Something will happen
- You can have multiple things happen.
- And catch any errors.

Can only go from Pending to Fulfilled or Rejected (no takebacks)

Example: 'I promise to return to your table'

- Pending: Waiting for my pizza
- Fulfilled: Pizza has arrived!!
- Rejected: Kitchen ran out of cheese. :(

Promises on MDN

Creating a Promise

function	description
`let promiseObj = new Promise(executorFn)`	Creates a new Promise object with the executorFn
`promiseObj.then(onFulfilled, onRejected)`	Invokes the onFulfilled (onRejected) function when the promise is fulfilled (rejected)
`promiseObj.catch(callback)`	Invokes the callback function if the promise is rejected (or an error occurs)

```
function executorFn(resolve, reject) {
    // ...
    if (conditionMet) {
        resolve(); // Passed by the Promise object
    } else {
        reject(); // Passed by the Promise object
    }
}
```

You define this function and pass it into the Promise constructor

Back to that Pizza

```
function orderExecutor(resolve, reject) { // reject not used here
    console.log('making our pizza...');
    setTimeout(resolve, 5000);
}

let orderPizza = new Promise(orderExecutor);
orderPizza.then(function () { console.log('eating pizza!'); });
```

Back to that Pizza

We can pass a value to resolve...

```
function orderExecutor(resolve, reject) {
    console.log('making our pizza...');
    setTimeout(function() {
        resolve('Here\'s your pizza!');
    }, 5000);
}

let orderPizza = new Promise(orderExecutor);
orderPizza.then(function (value) { console.log(value); });
```

That value gets passed to the function passed into `then`

Back to that Pizza

The functions passed to `then` can pass values to the next `then` callback

```
function eat(value) {
    return value + ', and now it\'s gone';
}

let orderPizza = new Promise(orderExecutor);
orderPizza.then(eat).then(function (value) { console.log(value); });
```

Back to that Pizza

You can also return other promises, which halt the execution of the next `then` callback until it's resolved

```
function eatExecutor(resolve, reject) {
    console.log('eating our pizza...');
    setTimeout(resolve, 3000);
}

function eat() {
    return new Promise(eatExecutor);
}

let orderPizza = new Promise(orderExecutor);
orderPizza.then(eat).then(function () { console.log('Paying the bill!'); });
```

Still Asynchronous

```
function orderExecutor(resolve, reject) {
    console.log('Pizza ordered...');
    resolve('Here\'s your pizza!');
}

let orderPizza = new Promise(orderExecutor);
orderPizza.then(function (value) { console.log(value); });
console.log('Waiting for my pizza!');
```

In what order do these log statements appear in the console?
Note that the `setTimeout` has been removed

Rejecting a Pizza

```
function orderExecutor(resolve, reject) { // here MUST have both parameters defined
    console.log('Pizza ordered...');
    setTimeout(function() {
        reject('Ran outta cheese. Can you believe it?');
    }, 2000);
}

let orderPizza = new Promise(orderExecutor);
orderPizza
    .then(function () { console.log('Woohoo, let's eat!'); })
    .catch(function (value) { console.log(value); });
```

`then` and `catch` Return Promises

```
function executor(resolve) {
    resolve('Woohoo!');
}

let myPromise = new Promise(executor);
let thenPromise = myPromise.then(console.log);
let catchPromise = thenPromise.catch(console.error);

console.log(thenPromise instanceof Promise); // true
console.log(catchPromise instanceof Promise); // true

console.log(myPromise === thenPromise); // false
console.log(myPromise === catchPromise); // false

console.log(thenPromise === catchPromise); // false
```

`then` and `catch` return *new* Promises

`then` and `catch` Return Promises

```
function executor(resolve) {
    resolve('Woohoo!');
}

function processStr(val) {
    // mellow out that message a bit
    return val.toLowerCase().replace('!', '');
}

let myPromise = new Promise(executor);
let thenPromise = myPromise.then(processStr);
console.log(thenPromise);
```

`processStr` returns a string, but `then` turns it into a Promise that immediately resolves with the value "woohoo"

`then` and `catch` Return Promises

```
function executor(resolve) {
    resolve('Woohoo!');
}

function processStr(val) {
    // mellow out that message a bit
    return new Promise(function(resolve) {
        resolve(val.toLowerCase().replace('!', ''));
    });
}

let myPromise = new Promise(executor);
let thenPromise = myPromise.then(processStr);
console.log(thenPromise);
```

The previous slide is equivalent to the previous

`then` and `catch` Return Promises

```
function executor(resolve) {
    resolve('Woohoo!');
}

function processStr(val) {
    // mellow out that message a bit
    return new Promise(function(resolve) {
        setTimeout(function() {
            resolve(val.toLowerCase().replace('!', ''));
        }, 5000);
    });
}

let myPromise = new Promise(executor);
let thenPromise = myPromise.then(processStr);
console.log(thenPromise);
```

Now, `thenPromise` is "PENDING" and won't resolve until the promise returned by `processStr` resolves.

Promises to the Rescue

This chaining of promises is what makes the below possible

```
orderPizza()
  .then(eat)
  .then(pay)
  .catch(badPizza);
```